

Relationship to ARL, L4 Witness, and Hardware Perimeter

AGL - Boundary Map and Layer Separation

Normative Support Layer

Status	Draft v0.1
Version	0.1
Layer	Relationship map / bound- ary clarification / cross- layer interpretation
Canonical home	ester-reality-bound
Parent layer	Actor_Grounding_Layer_v0.1.md

Kotov Ivan

Bruxelles, 2026

Contents

Abstract	2
1. Purpose	2
2. Scope	3
3. Non-goals	3
4. Why this separation is necessary	3
4.1 Witness theater	4
4.2 Review laundering	4
4.3 Perimeter reductionism	4
4.4 Boundary collapse	4
5. Four layers, four different questions	4
5.1 L4 Hardware Perimeter	4
5.2 Actor Grounding Layer (AGL)	5
5.3 Arbitration / Review Layer (ARL)	5
5.4 L4 Witness	5
6. Canonical relationship statements	6
6.1 Hardware Perimeter constrains feasibility	6
6.2 AGL constrains runtime reliance	6
6.3 ARL constrains procedural conflict	6
6.4 L4 Witness constrains auditability	6
7. Handoff order	6
7.1 Ordinary path	6
7.2 Conflict path	7
7.3 Commit-time discipline	7
8. No-substitution rules	7
8.1 Witness is not permission	7
8.2 Hardware Perimeter is not full grounding	7
8.3 ARL is not initiation qualification	7
8.4 AGL is not full dispute procedure	7
8.5 Presentation is not authority	8
9. Typical failure patterns if the layers are confused	8
9.1 “It was logged, therefore it was legitimate.”	8
9.2 “The node was in PROD mode, therefore the initiation was sound.”	8
9.3 “ARL reviewed it, therefore the source was grounded.”	8
9.4 “The source looked coherent, therefore it should have been allowed to act.”	8
9.5 “The dashboard makes it look orderly, so the path is halfway to permission.”	8
10. Why AGL belongs canonically in ERB, not ARL	9
11. Implementation-facing implications	9
11.1 Hardware Perimeter	9
11.2 AGL	9

11.3 ARL	9
11.4 L4 Witness.....	10
12. Interpretation priority if tension appears	10
13. Explicit bridge	10
14. Hidden bridges	10
14.1 Cybernetics / Ashby	10
14.2 Information theory / Cover & Thomas	11
15. Earth paragraph	11
16. Closing statement	11

Abstract

The Actor Grounding Layer (AGL) sits close to several neighboring layers that can be mistaken for it:

- **ARL** may look similar because both can stop a path.
- **L4 Witness** may look similar because both deal with traceability and validity.
- **Hardware Perimeter** may look similar because both care about reality at the edge of execution.

That similarity is useful, but dangerous.

If these layers collapse into one another, a system can begin to:

- treat logging as permission,
- treat hardware mode as full grounding,
- treat review as a substitute for initiation discipline,
- or ask conflict machinery to repair a path that should never have been allowed to move.

This document fixes the separation.

Its purpose is not to multiply terminology. Its purpose is to state, cleanly, what each layer does, where it begins, where it ends, and how they hand work to one another without laundering legitimacy.

1. Purpose

To define:

- how AGL differs from ARL,
 - how AGL differs from L4 Witness,
 - how AGL differs from L4 Hardware Perimeter,
 - how the four layers cooperate,
 - which layer answers which operational question,
 - what must happen first when a path moves toward consequence,
 - and how a system avoids collapsing qualification, review, proof, and physical control into one confused surface.
-

2. Scope

This document applies whenever a long-lived system must decide:

- whether a source may support runtime reliance,
- whether a disputed path should enter procedural review,
- whether an event is merely visible or actually authoritative,
- whether physical operating conditions are strong enough for sensitive execution,
- and how all of the above are represented without lying about what happened.

It applies especially to:

- local-first entities,
 - review-capable systems,
 - constrained domestic or institutional systems,
 - systems with witness-bound operation,
 - and systems expected to survive disagreement, degradation, and restart.
-

3. Non-goals

This document does **not**:

- replace the AGL core,
- restate ARL in full,
- restate L4 Witness schemas in full,
- restate Hardware Perimeter controls in full,
- define code,
- or declare that one layer can permanently absorb another.

It exists to prevent one kind of mistake:

asking the wrong layer to do the wrong job, then mistaking that confusion for sophistication.

4. Why this separation is necessary

Without explicit separation, four recurring pathologies appear.

4.1 Witness theater

The system logs a beautiful chain of events and acts as if the existence of a trace proves the source was fit to authorize action.

It does not.

4.2 Review laundering

The system sends an invalid or weakly grounded initiation into ARL and expects procedural review to retroactively dignify what should have been stopped earlier.

It should not.

4.3 Perimeter reductionism

The system assumes that because the node is in acceptable physical mode, the initiating actor, signal, or delegated source must therefore be grounded enough for reliance.

That does not follow.

4.4 Boundary collapse

The system treats one layer as universal: - witness as truth, - review as grounding, - hardware as legitimacy, - or grounding as a full replacement for dispute procedure.

That is how mature systems become structurally childish.

5. Four layers, four different questions

The cleanest distinction is by operational question.

5.1 L4 Hardware Perimeter

Question:

Is the node / room / physical execution surface in an acceptable operating mode for this class of action?

Primary concerns: - network mode, - radios, - power path, - maintenance state, - physical access, - anomaly flags, - circuit breaker posture, - operator attestation.

Hardware Perimeter answers a reality question about the execution habitat.

It does **not** answer whether the source itself is sufficiently grounded to authorize run-time reliance.

5.2 Actor Grounding Layer (AGL)

Question:

Is the initiating actor, source, or perception path grounded enough in real present execution state to support runtime reliance?

Primary concerns: - human/operator degradation, - local entity initiation state, - sensor/perception integrity in the live moment, - delegated/proxy/external-origin grounding, - revalidation at commit, - narrowing or denial under degraded grounding.

AGL answers a qualification question about source-state at the boundary where action may begin to matter.

It does **not** decide the full procedural dispute.

5.3 Arbitration / Review Layer (ARL)

Question:

Given a procedurally real conflict, what is the lawful review path, bounded outcome, and re-entry discipline?

Primary concerns: - standing, - evidence admissibility, - conflict class, - hold / freeze / quarantine, - review, - outcome, - appeal, - lawful re-entry or denial.

ARL answers a conflict-discipline question once a dispute exists in procedural form.

It does **not** replace the need to qualify whether the source should have been allowed to move the runtime toward that conflict in the first place.

5.4 L4 Witness

Question:

What happened, in what order, under what evidence envelope, and how can that sequence be checked later?

Primary concerns: - record integrity, - canonicalization, - chain continuity, - signatures, - envelopes, - control-point events, - replayable audit signal.

L4 Witness answers a traceability question.

It proves sequence and record structure. It does **not** by itself grant reliance, standing, permission, or release.

6. Canonical relationship statements

The relationship can be stated compactly.

6.1 Hardware Perimeter constrains feasibility

It says whether the execution environment is physically acceptable.

6.2 AGL constrains runtime reliance

It says whether the source is grounded enough to matter operationally.

6.3 ARL constrains procedural conflict

It says what happens when lawful review is required.

6.4 L4 Witness constrains auditability

It says whether the event trail can later be checked as an honest sequence.

This is the simplest stable map:

Perimeter = physical viability

AGL = source grounding

ARL = conflict procedure

Witness = traceability

7. Handoff order

The layers cooperate through handoff, not fusion.

7.1 Ordinary path

A normal sensitive path should read approximately as:

1. **Hardware Perimeter** confirms acceptable operating mode.
2. **AGL** qualifies the source for runtime reliance.
3. **Initiation gates** open, narrow, reroute, or deny.
4. **L4 Witness** records significant transitions.
5. If no conflict exists, the path continues under the permitted bounded scope.

7.2 Conflict path

A contested path should read approximately as:

1. **Hardware Perimeter** still constrains physical operating possibility.
2. **AGL** may already narrow, hold, or deny reliance.
3. If a procedurally real conflict exists, **ARL** takes the path into standing / admissibility / freeze / review / outcome.
4. **L4 Witness** records that this occurred.
5. Re-entry, if any, remains bounded and explicit.

7.3 Commit-time discipline

Even after ordinary initiation or ARL review, **AGL may still need to revalidate at commit**. A path can be acceptable at intake and ungrounded when consequence actually binds.

That is why “review passed once” is not enough for serious systems.

8. No-substitution rules

The following rules are normative by intent, even where the surrounding documents remain draft.

8.1 Witness is not permission

A logged event is not automatically a lawfully grounded event.

8.2 Hardware Perimeter is not full grounding

A safe room does not make a degraded actor or proxy-origin fit for ordinary reliance.

8.3 ARL is not initiation qualification

A clean review layer must not be forced to repair every invalid initiation path.

8.4 AGL is not full dispute procedure

Grounding qualification does not replace standing, evidence review, outcome discipline, or appeal logic.

8.5 Presentation is not authority

A visible branch, state, or relation does not become authoritative because one of the four layers can display or record it.

9. Typical failure patterns if the layers are confused

9.1 “It was logged, therefore it was legitimate.”

False. That only means the system preserved a trace.

9.2 “The node was in PROD mode, therefore the initiation was sound.”

False. The habitat may be acceptable while the actor is degraded.

9.3 “ARL reviewed it, therefore the source was grounded.”

False. ARL may have done its job after AGL should already have narrowed or denied initiation.

9.4 “The source looked coherent, therefore it should have been allowed to act.”

False. Runtime coherence without grounding is one of the most common forms of architectural self-deception.

9.5 “The dashboard makes it look orderly, so the path is halfway to permission.”

False. Graphs, logs, summaries, and branches may remain visible while still lacking standing, grounding, or lawful re-entry.

10. Why AGL belongs canonically in ERB, not ARL

AGL belongs canonically in ester - reality - bound because its central concern is not dispute as such.

Its concern is **present-state grounding at the boundary where action becomes possible**.

That is closer to:

- physical mode,
- sensor integrity,
- local state,
- operator condition,
- degraded channels,
- and fail-closed execution posture

than to the procedural logic of arbitration itself.

ARL needs AGL. ARL consumes the consequences of AGL. But ARL is not the natural conceptual home of that upstream layer.

11. Implementation-facing implications

This separation is not only theoretical.

It maps directly onto implementation work already visible in the ECC-facing bridge package.

11.1 Hardware Perimeter

Feeds grounding with: - anomaly flags, - PROD/MAINT mode, - network/radio posture, - circuit-breaker state, - physical access and operator attestation.

11.2 AGL

Consumes source-state and produces: - gate-open, - gate-open-with-limits, - gate-reroute, - gate-deny, - gate-quarantine, - commit-time revalidation decisions.

11.3 ARL

Consumes procedurally real conflicts and produces: - standing decisions, - evidence decisions, - freeze / quarantine, - outcomes, - appeal, - re-entry discipline.

11.4 L4 Witness

Records significant transitions across all of the above.

This is why a runtime hook map can exist without pretending one hook does everything.

12. Interpretation priority if tension appears

If future package text creates tension between these four layers, interpret in this order:

1. **Hardware impossibility and hard perimeter breaks remain non-negotiable**
2. **AGL may deny or narrow runtime reliance before ARL exists**
3. **ARL governs lawful conflict procedure once conflict is admitted**
4. **L4 Witness records what occurred but does not grant authority by itself**

This preserves a clean causal order.

13. Explicit bridge

Hardware Perimeter □ **Actor Grounding Layer** □ **ARL** □ **L4 Witness**

Physical viability constrains source grounding.

Source grounding constrains whether a path may rely, initiate, or escalate.

ARL constrains what happens once conflict is procedurally real.

Witness binds the resulting sequence into durable traceability.

14. Hidden bridges

14.1 Cybernetics / Ashby

These layers exist separately because one regulator cannot safely absorb all varieties at once. Physical mode, source-state, conflict procedure, and audit signal require different control surfaces. Collapsing them reduces variety exactly where serious systems need more of it.

14.2 Information theory / Cover & Thomas

Different layers carry different evidentiary bandwidth. Perimeter provides coarse physical state. AGL consumes live grounding signals. ARL consumes bounded procedural objects. Witness compresses the event into a portable chain. If one layer is forced to carry all meanings, the channel becomes noisy and ambiguous.

15. Earth paragraph

In a real airport, the runway condition, the pilot's fitness, the flight authorization, and the black-box recording are not the same layer just because all four matter before the plane leaves the ground. Dry runway does not prove the pilot is fit. A valid route does not prove the aircraft should depart now. A flight recorder does not authorize takeoff. Serious aviation survives by keeping those questions separate and connected. Long-lived AI systems will need the same adulthood.

16. Closing statement

The point of this document is not to add another acronym to the pile.

The point is to stop a recurring architectural lie:

that if a system can review, record, or display a path beautifully enough, it can therefore be trusted to act on that path.

It cannot.

First the environment must be viable. Then the source must be grounded. Then conflict, if real, may enter review. And all of it should leave an honest trail.

That order is not bureaucracy.

It is what separates bounded intelligence from polished drift.